OLK — Open Library Kashmir

A Technical & Product Deep Dive

OLK Engineering

July 7, 2026

Agenda — roughly 45 minutes

Why OLK exists — product & the constraint that shapes everything	5 min
A tour of the platform — what it does today, for the whole team	9 min
Tech stack at a glance — what it's built with, and why	5 min
Under the hood — architecture, database, security, GPS, deploy	18 min
The road ahead — mobile, self-hosting, scaling to 50k+ users	7 min
Discussion	1 min

One honest note before we start

This deck is a companion to the full 24-chapter technical manual at `Guide/main.pdf` — everything here traces back to a chapter there.

Why OLK Exists

What is OLK?

Open Library Kashmir is a community book-sharing platform for readers in the Kashmir valley.

- Someone with books on a shelf lists them — to lend (they get it back) or donate (they don't).
- Someone who wants to read finds them by browsing nearby listings and sends a request.
- The two arrange an in-person handover.

No shipping. No warehouses. No payments.

The entire transaction is a conversation between two neighbours and a walk to a bus stop.

The one design decision that explains everything else

Everything happens in person, within walking or short-commute distance.

That is not a missing feature — it is *the* product decision. It explains:

- why every book and every club carries a latitude/longitude *and* a plain-language area name;
- why the database has a distance function instead of a shipping-address field;
- why there is no payment table anywhere in the schema;
- why trust — ratings, warnings, bans, reports — matters as much as inventory.

Where we are today

Product

- Full exchange lifecycle, live
- Wishlists, clubs, ratings
- Admin & moderation subsystem
- Regionally real: 29 seeded Kashmir areas across 10 districts, 19 genres

Engineering

- 24 database tables, all RLS-protected
- 19 chronological migrations
- Local Docker stack mirrors production
- A 24-chapter internal technical manual

The founder's two goals, in order

First: serve real readers in Kashmir, live, on the web. Second: eventually run the platform's own infrastructure — including the database — on a physical server *in* Kashmir, not on rented foreign infrastructure.

A Tour of the Platform

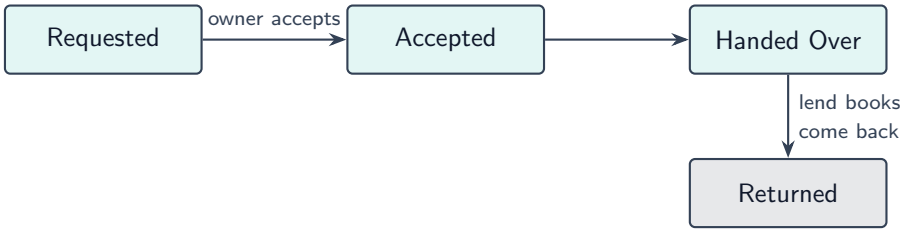
Discover & share a book

- **Browse** shows nearby books first — sorted by real distance, not just recency.
- Filter by genre, condition, listing type, and area.
- **List a book** in seconds: point your phone camera at the barcode.

The ISBN scanner

Scans the barcode, looks it up on the (free, open) Open Library catalogue, and auto-fills title, author, cover image, and even guesses a genre — conservatively: if it isn't confident, it leaves the field blank for you rather than guessing wrong.

From request to handover



- A reader requests; the owner accepts or declines.
- Owner marks it **handed over** when the two of you actually meet.
- Reading progress (0–100%) is visible to the whole community while it's out.

Two ways a book's story ends

Lending

- Book comes back — marked *Returned*
- Owner keeps it, can lend again

Donating

- "**Pass It On**" when the reader is done
- Ownership transfers — permanently
- The new owner can re-list it, and it can never quietly become a private copy again

One shared counter

Every book carries a `read_count` — incremented on *both* paths, so a donated book that's changed hands five times and a book that's been lent out five times both show up equally as "reached five readers."

Chat, rate, build trust

- Every request gets its own **realtime 1:1 chat** — conversations are organised by exchange, not by contact list.
- After a handover, both sides leave a **1–5 star rating**.
- Enough good ratings earns a public **"Trusted Sharer"** badge on your profile.

Why this matters as much as the books themselves

There's no escrow, no payment, no shipping carrier to blame if something goes wrong — reputation *is* the safety net.

Can't find a book? Wishlist it.

1. You add *"The Old Man and the Sea"* to your wishlist. Nothing happens yet.
2. Weeks later, someone — a total stranger — lists a copy, typed slightly differently.
3. The platform notices the match **automatically** and notifies you.

"Smart" matching, honestly explained

It's a fuzzy text-similarity search across *every* user's wishlist, not just yours — something that needs a deliberate, careful piece of backend engineering to do safely (Part IV).

Reading clubs

- Local, interest-based groups — each with its own location, so “nearby” applies to clubs too, not just books.
- Members post and discuss; membership and post counts stay accurate automatically.
- **Gated on trust**: you need 5+ completed exchanges and a clean record before you can start one.
- Deleting a club is always a soft deactivation — never an irreversible delete, even for the creator.

Staying informed

- Live **notification bell** — requests, messages, ratings, club posts, admin notices.
- Opt-out **weekly email digest** of new nearby books.
- Site-wide **announcements** banner and a curated **Book of the Month**.

One switch, whole platform

Admins can turn Clubs, Wishlists, Messages, or Ratings on or off for *everyone*, instantly — and put the entire site into maintenance mode — with no redeploy, no code change.

Keeping the community safe

- Anyone can **report** a user or a book, any time.
- Moderators triage: categorise, assign, resolve, or act in one click — ban, warn, hide.
- **Every** moderation action — who, what, why, when — is written to a permanent, tamper-evident audit log that not even a super-admin can edit or delete.

Tech Stack at a Glance

The stack, top to bottom

Layer	Technology	Role
Hosting	Vercel	Builds & serves the app; runs the weekly digest cron
Framework	Next.js 16 (App Router)	Routing, Server Components, Server Actions
UI runtime	React 19	Server/Client component model
Language	TypeScript (strict)	All application code
Styling	Tailwind CSS v4	Utility classes, no component library
Backend	Supabase	Postgres, Auth, REST API, Realtime, Storage
Local dev	Docker (via Supabase CLI)	The same images that could run self-hosted
Email	Resend	Transactional email & weekly digest

Deliberately small, on purpose

- **No ORM** — no Prisma, no Drizzle. Every query goes through Supabase's query builder or a raw Postgres function call.
- **No component library** — no MUI, Chakra, shadcn. Every button, card, and modal is hand-built.
- **No state-management library**, no React Query/SWR.

Why

This is a scope choice, not an oversight: keeping the dependency surface small enough that a couple of interns can hold the *entire* system in their heads.

Open source, top to bottom

- Next.js, React, Postgres, PostgREST, GoTrue (auth), Realtime, Storage — **every one** of these is open source and self-hostable.
- The only pieces that aren't: Vercel's hosting and Resend's email delivery — both are replaceable, swappable services sitting on top of an otherwise fully open stack.

Why this is the whole ballgame for Part V

Because Supabase itself is open source, "run our own server in Kashmir" is a realistic plan, not a fantasy — the exact containers running on a developer's laptop today are what would run on that server.

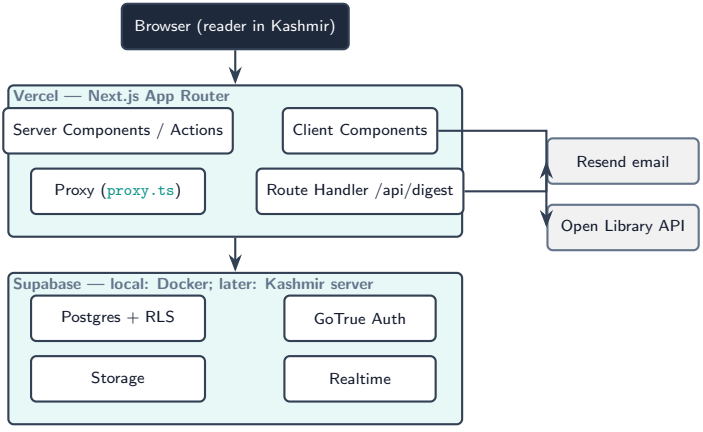
A warning for every new contributor

“This is NOT the Next.js you know.”

- Next.js 16 has real, breaking changes vs. most tutorials and training data.
- Concrete example: *Middleware* was renamed to *Proxy* — this project uses `src/proxy.ts`, not `middleware.ts`.
- The project’s own `AGENTS.md` tells every contributor (human or AI) to check `node_modules/next/dist/docs/` before writing framework-adjacent code.

Under the Hood

System architecture



The database box is Docker on a laptop today, and Supabase's hosted cloud in production — the long-term goal is for that same box to be physical hardware in Kashmir.

Server Components vs. Client Components

Every file under `src/app` is a **Server Component** unless its first line says `"use client"`.

- **Server**: renders on Vercel, talks to Postgres directly, ships *no* data-fetching code to the browser — the landing page and every layout.
- **Client**: hydrates and re-renders in the browser, talks to Supabase directly over HTTPS — subject to Row-Level Security *as the logged-in user*, never an elevated role. Nearly every interactive dashboard page.

An honest characteristic, not a recommendation

Most of this app's complex pages are Client Components today, even where a Server Component would be more efficient. That's a real, legitimate refactor to propose — not a rule someone already decided against.

The Proxy layer — one file, five jobs

`src/proxy.ts` runs on *every* request, before any page renders:

1. Refresh auth session cookies (unconditional, every request)
2. Read feature flags from `platform_settings` (cached 30s)
3. Redirect everyone but admins to `/maintenance` when maintenance mode is on
4. Redirect away from a disabled feature's whole route tree (clubs/wishlists/messages)
5. Gate unauthenticated, banned, and non-admin users off protected routes

This is the platform-wide switchboard behind the "one switch, whole platform" feature-flag slide from Part II.

Supabase fundamentals

"Supabase" is not one server — it's a bundle of independent, cooperating open-source services in front of stock Postgres:

- **Postgres** — the real database; every table is a plain, non-proprietary table.
- **PostgREST** — turns the schema into a REST API automatically. No hand-written backend for most operations.
- **GoTrue** — issues/verifies JWTs, owns `auth.users`.
- **Storage** — S3-like object store (book cover images).
- **Realtime** — streams row changes over WebSockets — powers chat and the notification bell.

The database, at a glance

24 tables, all in `public`, every one Row-Level-Security protected.

Identity & exchange

- `profiles`, `books`
- `book_requests`, `book_progress`

Community

- `messages`, `wishlists`
- `ratings`, `clubs` (+members, posts)
- `bookmarks`, `book_notes`

Notifications & content

- `notifications`, `announcements`
- `genres`, `areas`, `book_of_month`
- `platform_settings`

Admin & moderation

- `reports`, `user_bans/warnings`
- `admin_audit_log`, `admin_notes`

No ORM — the schema itself, not generated code, is the single source of truth.

auth.users vs. public.profiles

- Supabase Auth owns `auth.users` — email, password hash. Application code never touches it directly.
- `public.profiles` mirrors what the app actually needs (name, area, admin role, ban state).
- Kept in lock-step by **one database trigger**, fired the instant someone signs up.

A real, consequential exception

`books.owner_id` points at `auth.users`, not `profiles` — so fetching "the book owner's name" always needs a second, separate query. This is documented *in the code itself* as the single best piece of institutional memory in the whole frontend.

Row-Level Security — the real security model

- Every table: RLS **enabled**, default **closed**. No matching policy means no rows — for anyone, through any route.
- Postgres checks every query against `auth.uid()` / `auth.role()` from the caller's own JWT.
- For ordinary data access, **there is no separate authorization layer** in the application — the database *is* the security boundary.

Three admin roles, one hierarchy

`viewer` $\prec$ `moderator` $\prec$ `super_admin` — checked in exactly one place (`is_admin_or_mod()`, `is_super_admin()`), not duplicated across every policy that needs it.

`SECURITY DEFINER`: the escape hatch, used carefully

Some features *need* to cross the RLS boundary on purpose: matching your wishlist against a book someone else just listed means searching *everyone's* wishlist rows, not just your own.

- Solution: a narrow Postgres function that runs with elevated privilege — but only returns exactly the columns it was written to return.
- Discipline enforced throughout: never `SELECT *` on a sensitive table; explicit `REVOKE`/`GRANT` on the risky ones.

Why this matters

A bug in one of these functions is a bug that runs with elevated privilege — it is exactly as dangerous as it is useful.

How location actually works

- No PostGIS, no Google Maps API. Just two floats — `latitude`/`longitude` — on `profiles` and `clubs`, captured via the browser's native `navigator.geolocation` API.
- `area_name` is a free-text label shown *alongside* the coordinates — approximate, human, never an exact street address.

```
6371 * acos( cos(lat1)*cos(lat2)*cos(lng2-lng1)
           + sin(lat1)*sin(lat2) ) -- distance, in km
```

That's the entire "nearby" feature: the haversine great-circle formula, running as one Postgres function, sorting results by real distance. A clamp (`LEAST`/`GREATEST`) guards against a floating-point edge case that once made this return `NaN` for a user browsing their own listing.

Defense in depth: rate limiting & validation

Every important limit lives in the **database**, not just the UI — so it holds even against a direct API call:

- Max messages/hour and requests/day — enforced by `BEFORE INSERT` triggers, reading their limits from `platform_settings`.
- Club-creation eligibility (5+ completed exchanges, no reports) — checked client-side *and* re-checked by a database trigger.
- Free-text search is sanitised before being interpolated into the query filter DSL — the same category of defence as SQL parameterisation.

Docker & local development

- The *entire* Supabase stack — Postgres, GoTrue, PostgREST, Realtime, Storage, Studio — runs as Docker containers via the Supabase CLI. One command: `npx supabase start`.
- 19 migrations, replayed from scratch with `supabase db reset` any time local state is in doubt.
- Local and production are deliberately structured to be near-identical: same migrations, same RLS policies, same functions.

Not just developer convenience

Every hour spent comfortable with this Docker stack is rehearsal for the day it runs on real hardware in Kashmir instead of a laptop.

Deployment, today

- **Vercel** builds and serves the frontend — push to the connected branch, it redeploys automatically; PRs get their own preview.
- **Database migrations** are pushed separately: `supabase db push` against the hosted project — forward-only, no reset-and-replay safety net in production.
- The **weekly digest** is a real cron-triggered endpoint, authenticated with a shared secret — the one part of this app that's a genuine HTTP API rather than a page.

The service-role key that bypasses RLS entirely never reaches the browser — enforced at *build time*, not just by convention.

Honest accounting: what's still open

- Automated test coverage is minimal — a handful of unit tests, no end-to-end coverage yet.
- A few pages still use a plain browser `alert()` for error messages instead of the app's normal styled UI — known, tracked, not yet swept.
- Pending requests never expire automatically — nothing cleans up an old, unanswered one.

This is not a secret

All of this — and the reasoning behind every design decision in this section — lives in `Guide/main.pdf`, kept as the living source of truth, chapter by chapter.

The Road Ahead

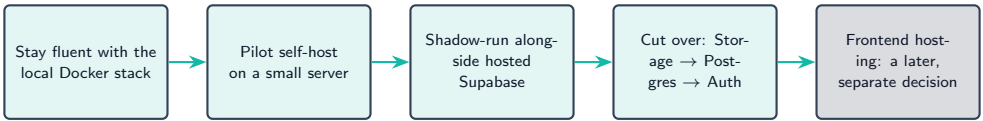
A mobile app, on the same backend

- Supabase's API is transport-agnostic — a React Native / Expo (or Flutter) app talks to the *exact same* Postgres, the same RLS policies, the same auth. No separate backend to build.
- Push notifications would sit naturally alongside — or eventually replace — the email digest.
- Worth designing for from day one: offline-first behaviour for areas with patchy connectivity.

Self-hosting in Kashmir — the long game

- Every Supabase service is open source and self-hostable — nothing about today's architecture is a rented black box.
- The Docker stack developers already run daily *is* the rehearsal for a physical server in-region.
- Why it matters: reducing dependency on foreign-hosted infrastructure, data sovereignty over community members' information, and long-term cost control.

Steps toward self-hosting



Lowest-risk service cuts over first (Storage), highest-risk last (Auth) — Vercel can keep serving the frontend throughout; that's a separate decision for later.

Scaling path: 0 → 50,000 users

- Today's architecture — Postgres + RLS + haversine queries — comfortably serves tens of thousands of users on a single modest Postgres instance. **No sharding needed yet.**
- First things to actually watch as usage grows: connection pooling (Supabase provides pgbouncer out of the box), indexes on hot paths (nearby search, area filters), and Storage bandwidth for cover images.
- Add basic observability — query stats, error tracking — *before* scale becomes a real problem, not after.

The headline

50,000 users is a tuning problem for this stack, not a rewrite.

More creative ideas, worth a conversation

- SMS/WhatsApp notifications for areas with limited smartphone data plans.
- Partnerships with schools and libraries — bulk onboarding, organised "book drive" events (the Book-of-the-Month and Announcements systems already exist to support campaigns like this).
- Multi-language UI — Kashmiri, Urdu, English.
- A public, read-only stats page — transparency about community impact.
- Community-run local "book hubs": a reading club paired with a physical drop-off point.

Thank you — let's get involved

- **Start here:** `Guide/main.pdf` — the full 24-chapter technical manual, written specifically for new maintainers.
- **Run it locally:** `npx supabase start && npm run dev`.
- Every design decision in this deck has a chapter, a migration, or a line of code behind it — nothing here was hand-waved.

Questions & discussion